

Software Architects in Large-Scale Distributed Projects

An Ericsson Case Study

Ricardo Britto and Darja Šmite, Blekinge Institute of Technology

Lars-Ola Damm, Ericsson

// This story is about Ericsson architects, their roles and responsibilities, and the effort they spent guarding and governing a large-scale product developed by distributed teams. Instead of trendy approaches, a centralized approach was necessary to deal with legacy and distribution. //



MUCH HAS BEEN written about software architects, their roles,^{1–5} their responsibilities,^{6–8} and how they're "made."⁹ Martin Fowler proposed this classification of software architects:⁴

- *Architectus Reloadus* makes important decisions early on, ensuring a system's conceptual integrity.
- *Architectus Oryzus* addresses problems in a project by

closely collaborating with the developers.

Philippe Kruchten summarized software architects' responsibilities, including defining the architecture; maintaining its architectural integrity; and consulting with, helping, and coaching development teams.¹ Paul Clements and his colleagues described architects' skills and knowledge, such as communication and learning skills.¹⁰ Grady Booch¹¹ and Frank Buschmann¹² have written on aspects of architects' daily work.

However, most of the vast literature on software architects has focused on the architect as a "one-soldier army." This seems reasonable in small projects but infeasible in large-scale projects, which require teams of architects.¹ Because many large-scale projects are spread across multiple locations, we raise three questions: What are software architects' roles and responsibilities? How are they organized in large-scale distributed software projects? How does adding new teams in remote locations influence the architects' role?

Here, we share our results from studying the development of a large, distributed software product at Ericsson, a global leader in telecommunications with sites worldwide. We focus on the architects steering this product's development.

Roles and Responsibilities

The architectural paradigm employed in a software product's architecture affects how software architects carry out their work.¹³ Whereas most new products at Ericsson have a modular or service-oriented architecture, legacy products, such as the case we describe here, have a monolithic architecture.

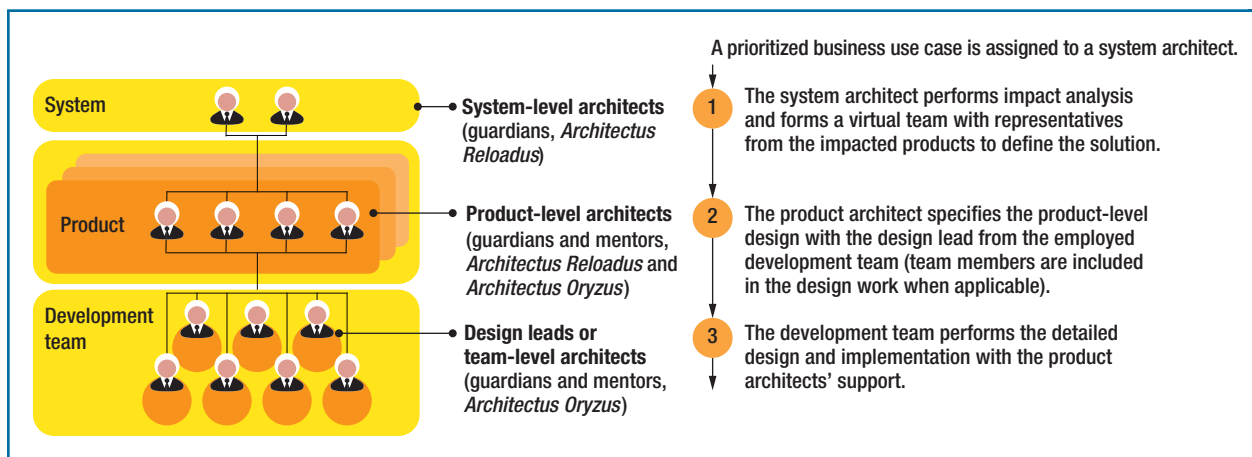


FIGURE 1. The organization and workflow of Ericsson's software architects. This centralized approach aims to ensure cross-team and cross-location alignment and to guard the complex system architectures.

The scale and complexity of Ericsson's software products, together with the distribution of development teams across multiple locations, also impact how architects perform their work. When teams and development locations are added to a project, it requires extra support to guard product integrity and mentor the teams while they climb the learning curve. In distributed environments, this introduces several challenges. One Ericsson software architect referred to communication overhead: "It would be much easier to explain to the developers things about the product's architecture if they were in Sweden, because most of the time we only communicate via the code review tool and email." Another architect pointed out the necessity of guarding and control: "When multiple teams work together on a task, the developers across teams do not communicate as well as within a team. Therefore, they often introduce task conflicts that we find in their code, which otherwise the teams would not identify themselves."

To ensure cross-team and cross-location alignment and to guard the

complex system architectures, Ericsson architects are organized in three levels (see Figure 1).

At the *system level*, a team of architects of the type *Architectus Reloadus* ensures that the products included in a system can communicate with each other. They closely collaborate with product-level architects and coordinate the design of the features, interfaces, and protocols used in the system—that is, the system architecture. They also review product-level solutions and guard the system's evolvability. These architects are mostly in Sweden.

At the *product level*, a team of architects fulfils the *Architectus Reloadus* and *Architectus Oryzus* roles. These architects, who are experienced, skillful developers, lead the design of technical solutions and guard the integrity and evolvability of the product and the system architecture. They collaborate closely with system-level architects and software development teams throughout solution implementation. They steer software design, ensure that design rules are followed, promote the use of design patterns, and ensure code

quality through code reviews. They help guard the architecture and the deliverables' quality. They also mentor the software teams by sharing their knowledge, providing feedback through code reviews, suggesting improvements, and helping out in urgent situations to speed up development. These architects are also mostly in Sweden, and they're this article's main focus.

At the *development team level*, senior developers called *design leads* (or team-level architects) support the design of technical solutions. They ensure that their team follows the design rules and design patterns, which generally are defined by product-level architects. They also help product-level architects guard a product's architectural integrity and evolvability and help ensure the quality of deliverables through code and documentation reviews. The more expertise and experience design leads have, the more autonomous their teams are; they can provide more support to product-level architects with mentoring and guarding.

The product and development team levels might vary slightly among

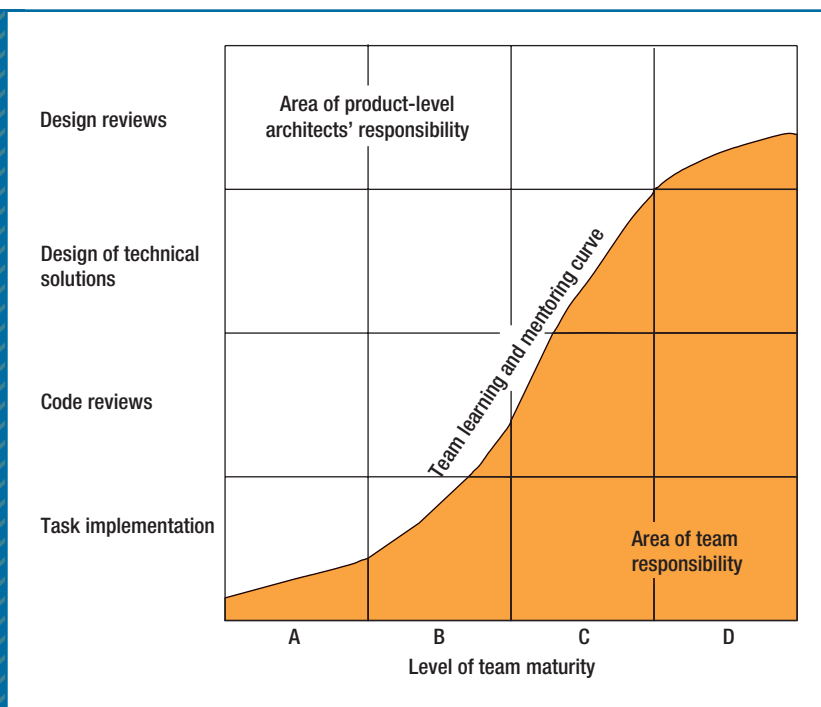


FIGURE 2. The authority matrix. For a description of the maturity levels, see the main article. As a team matures, it should get more independent and require less support from the product-level architects.

different Ericsson products. In our case, the product level was “thicker” and centralized almost all responsibility for the overall design of the technical solutions to be implemented in the product. The overall responsibility for guarding the product architecture’s integrity and evolvability was also centralized. This means that even the more experienced teams were never fully autonomous. More recent products with a service-oriented architecture have a much thinner product level. That is, they have approximately 50 percent fewer architects, and the teams shoulder much of the responsibility.

How Many Product-Level Architects Do You Need?

In our case study, the main factor affecting the extent of the product-level

architects’ involvement was the development teams’ maturity. To visualize the product-level architects’ amount of work and area of responsibility, we designed an authority matrix (see Figure 2). The curve in the matrix shows that as a team matures, it should get more independent and require less support from the product-level architects. The four activities on the y-axis are those that concerned the product-level architects the most. The x-axis indicates the four levels of team maturity.

At level A, an immature team has no or very basic knowledge about the product’s code and architecture. Level-A teams require much mentoring and support from the architects, even when implementing (coding, testing, and documenting) noncomplex technical solutions.

Product-level architects guide the teams during the implementation and are actively involved, with both reviewing and approving code.

At level B, teams have sufficient knowledge to implement noncomplex technical solutions without much guidance. More knowledgeable team members and design leads review the implemented code, but product-level architects still review most of the design and code.

At level C, experienced, mature teams with good knowledge of the product architecture implement complex technical solutions that have significant architectural impact. These teams review and approve their own code. Product-level architects are involved only in approval when the architecture’s critical components are affected. (A critical component contains core functionality that executes key operations.) Level-C teams also perform more solution design. Product-level architects support these teams by mentoring the design of technical solutions and providing on-demand guidance in the initial implementation stages.

At level D, very experienced and rather autonomous teams implement complex technical solutions independently, even ones affecting the product architecture’s critical components. The code they implement doesn’t need product-level architects’ approval. Level-D teams also drive the design and review of technical solutions. Product-level architects are involved only in the technical-solution design.

Our authority matrix is based on the product-level architects’ experience and the data we collected and analyzed in our case study (see the sidebar “The Ericsson Case Study” and Figure 3). In particular,



THE ERICSSON CASE STUDY

Our case study (see the main article) investigated the development of a telecom product that has evolved for more than 15 years. At the time of our investigation, the product employed more than 150 employees (including 15 product-level architects, 20 design leads, 100 developers working in 22 teams, and 15 other supporting roles) in Sweden, India, Italy, the US, and Turkey. One team comprised only product-level architects; each of the other teams consisted of a design lead and five to eight developers.

The results presented here are based on the data from 240 tasks carried out from January 2014 to June 2016. We employed the following data collection methods.

First, we performed archival research. We analyzed managerial documents (plans and reports) related to 26 product customization tasks (PCs), measuring the effort architects expended for development and support. Twelve teams participated in these PCs (for more on these teams, see the main article). We also surveyed the Gerrit code review repository to determine the number of defects identified through code reviews completed for a random selection of tasks: 10 PCs, 188 trouble reports (TRs), 9 product improvements (PIs), and 11 standardizations of market features (MFs). We manually categorized the defects using the ISO/IEC 25010:2011 standard.¹ The product's unit manager validated the collected archival data.

Next, we conducted unstructured interviews with the unit manager and product-level architects. We collected their knowledge about how the architects were organized, the architects' roles and responsibilities, and the development teams' maturity.

Finally, we held four workshops with the unit manager and product-level architects to measure the 26 PCs' task complexity—the difficulty of performing them. We selected one of the PCs as the baseline and had participants measure the other PCs in relation to it. Using planning poker, they attributed a positive integer (complexity points) to each PC. To check the assigned measures, the architects consulted technical-solution specifications whenever needed.

The PCs, PIs, and MFs resembled independent projects because they had start and end dates, a responsible team, and expected results. In general, PCs took 1 to 3 months, PIs took 1 to 2 months, and MFs took 2 to 6 months.

Reference

1. ISO/IEC 25010:2011—*Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—System and Software Quality Models*, Int'l Org. for Standardization, 2011; www.iso.org/iso/catalogue_detail.htm?csnumber=35733.

our statistical analysis of the performance data confirmed that the more immature teams required more support from the product-level architects.

Figure 3a shows the teams' distribution. The Turkish team was classified as level A. The five recently onboarded Indian teams started at level A; four of them progressed to level B during our study. The four Swedish teams and the US team were classified as level C. One Swedish team temporarily reached level D (product-level architects joined the team), and a temporary level-D team comprised product-level architects, developers, and design leads from Sweden and India.

The Effort Needed to Guard and Mentor

The teams' maturity also determined the number of required product-level architects. In our case study, the rule of thumb seemed to be that each level-C team needed approximately one-half of a product-level architect and that each level-A or level-B team needed one product-level architect. As we mentioned before, no permanent level-D teams were in our case study. In cases involving critical work demanding specific architectural knowledge and quick development, product architects might temporarily join a team to engage in coding and testing. Such teams can qualify as level-D teams.

Figure 3b shows the effort that product-level architects spent supporting 12 teams (the teams we mentioned earlier, except the temporary Swedish team) performing 26 product customization tasks. The less mature teams (levels A and B) received significant support from the product-level architects. In contrast, mature teams (levels C and D) worked more autonomously. We verified these results' statistical significance using the Kruskal-Wallis one-way analysis of variance (significance level = 0.01, p -value = 0.008493).

Part of the effort involved mentoring, which the product-level architects provided through meetings

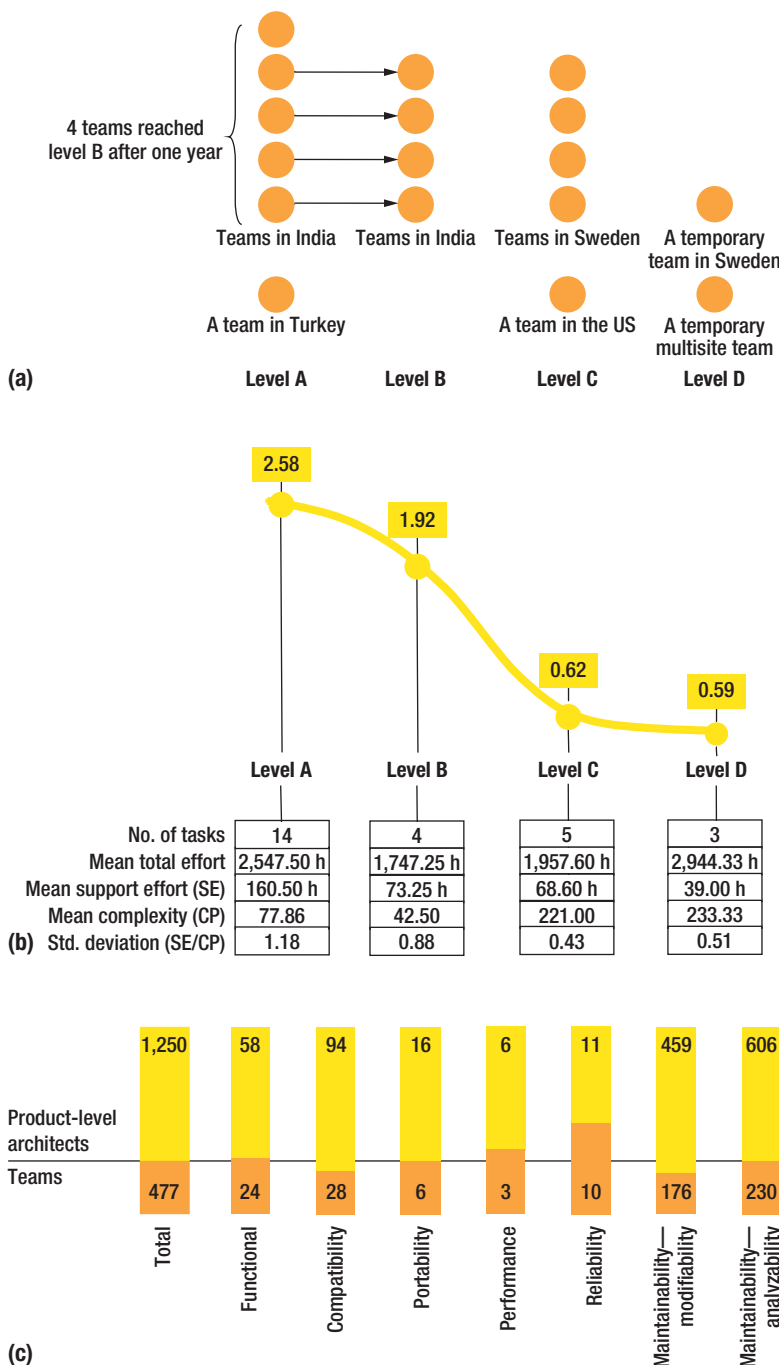


FIGURE 3. Evidence supporting the authority matrix in Figure 2. (a) The distribution of the teams in our case study. (b) The effort the architects spent supporting the teams. The curve shows the mean ratio between the support effort and task complexity. (c) The proportion of different defects the product-level architects and the teams found. The more immature teams required more support from the product-level architects.

(videoconferences or teleconferences with remote teams) or synchronous chat. Most of the effort involved code reviews, which also provided mentoring opportunities and involved clarification of architectural principles and improvement recommendations. In some cases, support included detailed instructions that even contained code samples to facilitate the understanding of the design.

Problems That Product-Level Architects Prevented

To ensure the product architecture's integrity and evolvability, product-level architects review new and modified code to identify defects that might harm the product architecture, induce technical debt, and complicate the architecture's evolvability. Although Ericsson devotes much attention to rigorous regression tests, not all defects can be caught automatically, especially non-functional maintainability defects.¹⁴ Maintainability defects can lead to code entropy,¹⁵ affecting the architecture's integrity.

At Ericsson, new developers working on legacy products often have problems understanding the code's structure, which leads to the fear of changing existing code and the tendency to duplicate code.¹⁶ This behavior is common in immature teams, and architects play a vital role in detecting such defects through code reviews, making code review a crucial asset.

Figure 3c contains results about defects revealed through code reviews performed on 220 tasks. We focused on two subcategories of maintainability defects closely related to Ericsson architects' guardian role: modifiability and analyzability.¹⁷

The code reviews revealed 1,727 defects; product-level architects dis-

covered 72.38 percent of those defects, and the teams discovered 27.62 percent. Of those defects, 85.18 percent were maintainability defects, and the product-level architects revealed 72.40 percent of them. Considering that testing techniques don't easily reveal maintainability defects, the product-level architects' active participation helped in early detection of defects that otherwise could have affected the product architecture's integrity and evolvability.

Although the software architects' hierarchy, roles, and responsibilities we described aren't unusual,^{1,18} they aren't trendy, either. So, in this case study, why didn't Ericsson fully embark upon trendier approaches such as microservices¹⁹ and agile ways of working?

The answer is, because of legacy code and distributed development. Although this case study implemented many agile practices such as continuous integration, continuous

delivery, and DevOps, certain limitations prevented fully exploiting the new approaches.

The product's monolithic architecture made relying on coordination by mutual adjustment problematic.²⁰ However, this doesn't automatically mean that Ericsson applied traditional coordination and control. The architects' hierarchy wasn't traditional in the sense that people higher up in the hierarchy could overrule the ones further down. Rather, it was a network of architects with

CSI-IEEE CS Joint Education Award

CALL FOR AWARD NOMINATIONS
Deadline 15 October 2016

► CRITERIA

The Computer Society of India (CSI) / IEEE Computer Society Education Award is a joint CSI-IEEE national society award that recognizes educators who have made significant contributions to computer science and engineering education.

► ELIGIBILITY

- Nominees should be teaching in India, preferably for a least the past three years.
- Nominators must be either CSI, IEEE or IEEE CS members.
- Self-nominations are not accepted.

► NOMINATION REQUIREMENTS

The nomination form must include at least two endorsements.

► AWARD

The CSI/IEEE-CS Education Award will consist of a certificate and US \$500 honorarium.

► PRESENTATION

The award will be presented at the CSI 2016: Annual Convention of CSI to be held between 8–10 December 2016 at Le Méridien Coimbatore, India.

► SUBMISSIONS

Nomination forms should be sent to csiieee@csi-india.org by the 15 Oct. deadline.



IEEE  computer society



THE PRESENT AND FUTURE OF SOFTWARE ARCHITECTS AT ERICSSON

The speed with which new technologies are introduced has compelled Ericsson's software architects to adapt quickly. For example, cloud computing forces architects to rethink existing architectures and use different approaches to design new ones. It's no longer only about designing software to conform to a particular hardware specification, because virtual environments hide the underlying infrastructure.

At the same time, many new technologies and methods have reduced Ericsson architects' workload. For example, continuous-integration and continuous-delivery tools require less manual effort to ensure that code changes don't harm a product architecture's behavior. To implement continuous integration and continuous delivery in our case study (see the main article), we adapted the product's architecture to maintain full backward compatibility despite the frequent releases—for example, through strict version control of interfaces.

Ericsson also employs DevOps. In our case study, two product-level architects were dedicated to ensuring smooth collaboration between Ericsson's development team and one customer's operation team. This shortened feedback loops, which led to better interaction with Ericsson's customers.

Another trend influencing the role of Ericsson architects is the agile transformation. To be able to benefit from agile ways of working, teams are expected to have more autonomy, and architects serve primarily as mentors supporting the teams. Although Ericsson has tried to increase team autonomy for the case study's product, the company has focused on increasing autonomy for more recent products not limited by legacy architectures.

Finally, Ericsson architects must be prepared to work on products developed at multiple locations. As in many other large organizations, corporate cultures and ways of working differ among Ericsson's distributed units. So, software architects are often expected to possess strong leadership skills and the ability to align the philosophy, traditions, and attitudes toward quality and maintainability across all development locations.

different focus areas who were involved in decisions related to their competences. On the product level, the architects and other experienced developers had designated approval rights based on their competences. They governed this approval structure and process as they saw fit.

This governance structure was similar to large distributed open source projects such as Eclipse and Android. That is, the open-source

community has also concluded that large development projects sometimes require a centralized approach to secure the software's quality.

Finally, the architects' hierarchy was also an answer to the challenges of distribution. Ericsson is no exception to the trend of offshoring parts of development to low-cost regions; much of its product development has become global. However, many offshore sites are infamous for high

attrition rates. This means that employee turnover results in continuous loss of expertise and the never-ending demand for mentoring and steering. Knowledge loss also occurs from frequently moving resources in and out of products because of rapidly changing business needs. In such situations, the centralization of architectural decisions such as we described here secures the architectural knowledge's stability and its availability for new developers and teams.

For a look at the present and future of Ericsson software architects, see the related sidebar. 🔍

References

1. P. Kruchten, "The Architects—the Software Architecture Team," *Proc. 1st Working IFIP Conf. Software Architecture* (WICSA 99), 1999, pp. 565–583.
2. J.F. Hoorn et al., "The Lonesome Architect," *J. Systems and Software*, vol. 84, no. 9, 2011, pp. 1424–1435.
3. M.B. Romney, "The Software Architect," *Comm. ACM*, vol. 50, no. 5, 2007, pp. 75–81.
4. M. Fowler, "Who Needs an Architect?," *IEEE Software*, vol. 20, no. 5, 2003, pp. 11–13.
5. S. Sherman and I. Hadar, "Toward Defining the Role of the Software Architect," *Proc. 8th Int'l Workshop Cooperative and Human Aspects of Software Eng.* (CHASE 15), 2015, pp. 71–76.
6. P. Kruchten, "What Do Software Architects Really Do?," *J. Systems and Software*, vol. 81, no. 12, 2008, pp. 2413–2416.
7. A.B. Bondi, "The Software Architect as the Guardian of System Performance and Scalability," *Proc. ICSE Workshop Leadership and Management in Software Architecture* (LMSA 09), 2009, pp. 28–31.
8. A. Tang et al., "On the Responsibilities

- of Software Architects and Software Engineers in an Agile Environment: Who Should Do What?," *Proc. 4th Int'l Workshop Social Software Eng.* (SSE 11), 2011, pp. 11–18.
9. B. Correa, "How Are Architects Made?," *IEEE Software*, vol. 30, no. 5, 2013, pp. 11–13.
 10. P. Clements et al., "The Duties, Skills, and Knowledge of Software Architects," *Proc. 6th Working IEEE/IFIP Conf. Software Architecture* (WICSA 07), 2007, pp. 20–22.
 11. G. Booch, "The Architect's Journey," *IEEE Software*, vol. 28, no. 3, 2011, pp. 10–11.
 12. F. Buschmann, "Value-Focused System Quality," *IEEE Software*, vol. 27, no. 6, 2010, pp. 84–86.
 13. J. Lewis and M. Fowler, "Microservices: A Definition of This New Architectural Term," 25 Mar. 2014; <http://martinfowler.com/articles/microservices.html>.
 14. M.E. Fagan, "Design and Code Inspections to Reduce Errors in Program Development," *IBM Systems J.*, vol. 15, no. 3, 1976, pp. 182–211.
 15. C.E. Ebeling, *An Introduction to Reliability and Maintainability Engineering*, 2nd ed., McGraw-Hill, 2009.
 16. G. Hanssen et al., "Software Entropy in Agile Product Evolution," *Proc. 43rd Hawaii Int'l Conf. System Sciences* (HICSS 10), 2010, pp. 1–10.
 17. *ISO/IEC 25010:2011—Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—System and Software Quality Models*, Int'l Org. for Standardization, 2011; www.iso.org/iso/catalogue_detail.htm?csnumber=35733.
 18. J.Y. Bang et al., "How Software Architects Collaborate: Insights from Collaborative Software Design in Practice," *Proc. 6th Int'l Workshop Cooperative and Human Aspects of*

ABOUT THE AUTHORS



RICARDO BRITTO is a PhD candidate in the Blekinge Institute of Technology's Department of Software Engineering. His research interests include large-scale agile software development, global software engineering, search-based software engineering, and software process improvement. Britto received an MSc in electrical engineering from the Federal University of Rio Grande do Norte. Contact him at rbr@bth.se.



DARJA ŠMITE is a professor in the Blekinge Institute of Technology's Department of Software Engineering and a part-time research scientist at SINTEF Information and Communication Technology. Her research interests include the effect of globalization and offshoring on software companies, large-scale agile software development, and software process improvement. Šmite received a PhD in computer science from the University of Latvia. Contact her at darja.smite@bth.se.



LARS-OLA DAMM works at Ericsson, where he manages software development teams working on large distributed projects. His main research interest is software process improvement. Damm received a PhD in software engineering from the Blekinge Institute of Technology. Contact him at lars-ola.damm@ericsson.com.

- Software Eng.* (CHASE 13), 2013, pp. 41–48.
19. A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices Architecture Enables DevOps," *IEEE Software*, vol. 33, no. 3, 2016, pp. 42–52.
 20. H. Mintzberg, *Mintzberg on Management: Inside Our Strange World of Organizations*, Free Press, 2007.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.

IEEE
SECURITY & PRIVACY



@securityprivacy